OASIS INFOBYTE

DATA ANALYTICS INTERNSHIP

LEVEL 2 – TASK 3

FRAUD DETECTION

Final Project Report

Submitted By

Madala Sai Sprisha

Internship

OASIS INFOBYTE – Data Analytics Internship

Domain

Data Analytics / Machine Learning

Project Title

Level 2 – Task 3: Fraud Detection

Technologies Used

Python • Pandas • NumPy • Matplotlib • Seaborn • Scikit-learn • Jupyter Notebook

• Imbalanced-learn

Submitted To

OASIS INFOBYTE

Year

2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to OASIS INFOBYTE for providing me with the opportunity to complete Level 2 – Task 3, Fraud Detection. This project provided practical exposure to exploratory data analysis, data-quality assessment, class-imbalance analysis, feature preparation, train-test splitting, feature scaling, SMOTE-based resampling, classification modelling, evaluation metrics, confusion-matrix analysis, ROC-AUC comparison, and feature-importance interpretation.

I also thank the mentors and coordinators for providing the internship task requirements and learning resources. The structured workflow helped me understand why fraud detection requires evaluation measures beyond accuracy when the target classes are highly imbalanced.

The implementation was completed in a Jupyter Notebook using the Credit Card Fraud Detection Dataset. The workflow begins with dataset loading and inspection, continues through transaction amount and time-based exploratory analysis, addresses class imbalance using SMOTE on the training data, and ends with Logistic Regression and Random Forest model evaluation.

This work strengthened my practical understanding of supervised machine learning for fraud detection and demonstrated the importance of class balance, recall, precision, F1-score, ROC-AUC, and interpretable model comparison.

ABSTRACT

This project focuses on detecting fraudulent financial transactions using machine-learning classification techniques on a highly imbalanced credit-card transaction dataset. The executed notebook contains 284,807 transactions and 31 columns, including Time, 28 anonymized V1–V28 features, Amount, and the binary Class target. Only 492 transactions are fraudulent, representing 0.1727% of the complete dataset, while 99.8273% are non-fraudulent.

The workflow begins with dataset loading, structural inspection, descriptive statistics, missing-value analysis, class-distribution analysis, transaction-amount visualization, and time-of-day analysis. The Time variable was transformed into an Hour feature for exploratory analysis. The feature matrix contains 30 predictors after excluding Class and the derived Hour variable. An 80:20 stratified train-test split produced 227,845 training observations and 56,962 testing observations. StandardScaler was fitted on the training data and applied to both subsets. SMOTE was then applied only to the training data, balancing the two classes at 227,451 samples each.

Two classifiers were trained: Logistic Regression and Random Forest. Logistic Regression achieved 97.41% accuracy, 5.78% precision, 91.84% recall, 10.88% F1-score, and 0.9708 ROC-AUC. Random Forest achieved 99.88% accuracy, 62.12% precision, 83.67% recall, 71.30% F1-score, and 0.9729 ROC-AUC. The Random Forest model therefore provided the stronger balance between precision and recall. Its feature-importance analysis identified V14, V10, V4, V17, and V3 among the most influential predictors.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The Fraud Detection project focuses on building a supervised machine-learning pipeline to identify fraudulent financial transactions from a highly imbalanced dataset. The project uses transaction-level attributes, including anonymized numerical variables, transaction time, transaction amount, and the Class target, to distinguish fraudulent transactions from legitimate ones.

The implementation uses Python, Pandas and NumPy for data handling, Matplotlib and Seaborn for visualization, Scikit-learn for preprocessing, modelling and evaluation, and Imbalanced-learn for SMOTE-based class balancing. The complete workflow is documented and executed in a Jupyter Notebook.

1.2 Problem Statement

Fraudulent transactions form only a very small fraction of the complete transaction population. In the executed dataset, fraud accounts for just 0.1727% of transactions. This severe class imbalance makes ordinary accuracy an unreliable measure because a classifier can obtain a very high accuracy by predicting most transactions as non-fraudulent while still missing many fraudulent cases.

The project therefore aims to develop and compare classification models using metrics that better reflect fraud-detection performance, especially precision, recall, F1-score, and ROC-AUC.

1.3 Project Objectives

1. Load and inspect the Credit Card Fraud Detection Dataset.
2. Assess dataset structure, descriptive statistics, and missing values.
3. Analyze class imbalance and visualize transaction amounts and time-based patterns.
4. Create an Hour feature from transaction Time for exploratory analysis.
5. Prepare the feature matrix and binary fraud target.
6. Apply an 80:20 stratified train-test split.
7. Standardize the features using StandardScaler.
8. Apply SMOTE only to the training data to address severe class imbalance.
9. Train Logistic Regression and Random Forest classifiers.
10. Evaluate the models using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices.

11. Compare the models and identify the most influential Random Forest features.

12. Discuss the most important evaluation metric for fraud detection and the scalability of a high-volume deployment.

CHAPTER 2: DATASET & TOOLS

2.1 Dataset Description

The project uses the Credit Card Fraud Detection Dataset. The executed notebook reports 284,807 rows and 31 columns. The columns consist of Time, 28 anonymized variables named V1 through V28, Amount, and Class. The Class variable is binary, where 0 represents a non-fraudulent transaction and 1 represents a fraudulent transaction.

The dataset contains 284,315 non-fraudulent transactions and 492 fraudulent transactions. The fraud class therefore represents 0.1727% of all observations, confirming the severe imbalance that motivates the use of SMOTE and fraud-focused evaluation metrics.

2.2 Tools & Technologies Used

TOOL / TECHNOLOGY	PURPOSE
Python	Programming language and machine-learning workflow
Pandas	Dataset loading, inspection, transformation, and tabular analysis
NumPy	Numerical operations and supporting calculations
Matplotlib	Plotting and visual analysis
Seaborn	Statistical visualization and heatmaps
Scikit-learn	Train-test split, scaling, classification models, and evaluation
Imbalanced-learn	SMOTE-based oversampling of the training data
Jupyter Notebook	Interactive implementation and documentation
CSV	Input dataset storage

2.3 Project Folder Structure

The implementation uses a dataset folder containing the credit-card CSV, a notebook containing the complete Python workflow, and a screenshots/report area for preserving visual evidence and documentation. The notebook loads the dataset through the relative path ../dataset/creditcard.csv.

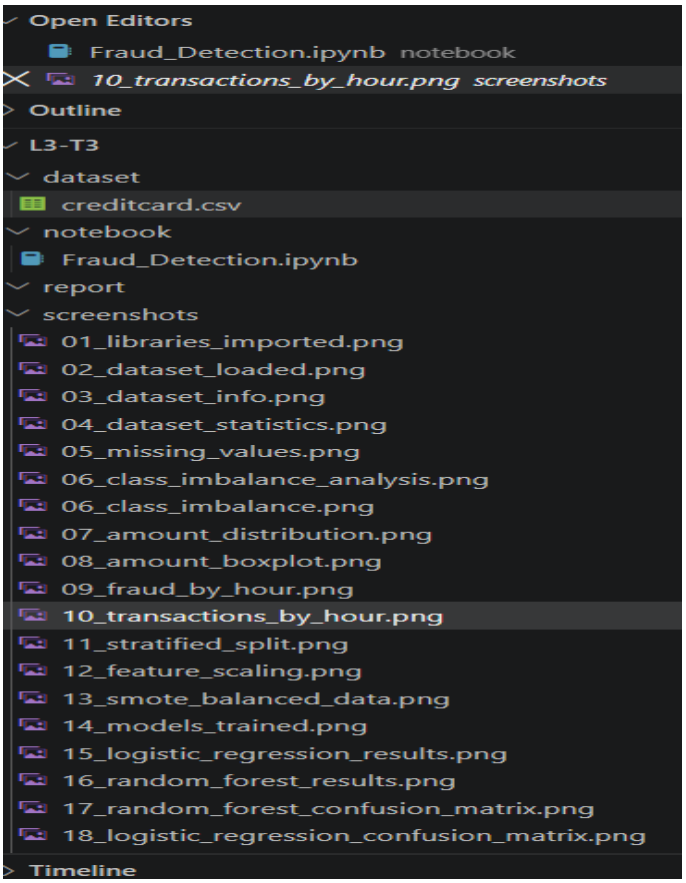

Figure 2.1: Project Folder Structure of the Fraud Detection Project

2.4 Input Dataset

The input dataset is the Credit Card Fraud Detection Dataset stored as creditcard.csv. It contains 284,807 transaction records. The dataset includes a Time column, anonymized V1–V28 variables, Amount, and the Class target. The first rows shown in the notebook confirm the numerical structure of the transaction data.

```
Notebook Output – Cell 3
Dataset loaded successfully!
Dataset Shape: (284807, 31)

   Time        V1        V2        V3        V4        V5        V6        V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3   1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4   2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941

         V8        V9  ...       V21       V22       V23       V24       V25  \
0  0.098698  0.363787  ... -0.018307  0.277838 -0.110474  0.066928  0.128539
1  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288 -0.339846  0.167170
2  0.247676 -1.514654  ...  0.247998  0.771679  0.909412 -0.689281 -0.327642
3  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321 -1.175575  0.647376
4 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458  0.141267 -0.206010

        V26       V27       V28  Amount  Class
0 -0.189115  0.133558 -0.021053  149.62      0
1  0.125895 -0.008983  0.014724    2.69      0
2 -0.139097 -0.055353 -0.059752  378.66      0
3 -0.221929  0.062723  0.061458  123.50      0
4  0.502292  0.219422  0.215153   69.99      0

[5 rows x 31 columns]
```

Figure 2.2: Initial Dataset Loading and Preview

Interpretation:

The dataset was loaded successfully with 284,807 observations and 31 columns. The preview contains numerical transaction attributes, Amount, and the binary Class target, making the data suitable for numerical preprocessing and classification.

CHAPTER 3: FRAUD DETECTION IMPLEMENTATION

3.1 Library Import & Dataset Loading

The required Python libraries were imported successfully, including Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn utilities and Imbalanced-learn. The credit-card CSV was then loaded into a Pandas DataFrame using pd.read_csv().

```
Notebook Output — Cell 3
Dataset loaded successfully!
Dataset Shape: (284807, 31)

   Time        V1        V2        V3        V4        V5        V6        V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3   1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4   2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941

         V8        V9  ...       V21       V22       V23       V24       V25  \
0  0.098698  0.363787  ... -0.018307  0.277838 -0.110474  0.066928  0.128539
1  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288 -0.339846  0.167170
2  0.247676 -1.514654  ...  0.247998  0.771679  0.909412 -0.689281 -0.327642
3  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321 -1.175575  0.647376
4 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458  0.141267 -0.206010

        V26       V27       V28  Amount  Class
0 -0.189115  0.133558 -0.021053  149.62      0
1  0.125895 -0.008983  0.014724    2.69      0
2 -0.139097 -0.055353 -0.059752  378.66      0
3 -0.221929  0.062723  0.061458  123.50      0
4  0.502292  0.219422  0.215153   69.99      0

[5 rows x 31 columns]
```

Figure 3.1: Dataset Loading and Initial Preview

Interpretation:

The output confirms successful dataset loading and shows the first transaction records. The dataset shape is 284,807 rows by 31 columns.

3.2 Data Inspection & Quality Assessment

The DataFrame structure was inspected to determine the number of records, columns, non-null counts, and data types. All 31 columns contain 284,807 non-null values. Thirty columns are float64 variables and Class is an integer variable.

```
Notebook Output — Cell 5
Number of rows: 284807
Number of columns: 31

Column names:
['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10', 'V11', 'V12',
'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23', 'V24',
'V25', 'V26', 'V27', 'V28', 'Amount', 'Class']

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

Figure 3.2: Dataset Information and Data Types

Interpretation:

The complete non-null counts indicate that the dataset has no missing observations and has a fully numerical structure, which simplifies preprocessing.

3.3 Descriptive Statistics

Descriptive statistics were generated using df.describe(). The output provides count, mean, standard deviation, minimum, quartiles, and maximum values for the transaction variables. The anonymized V features are centered close to zero, while Time and Amount have their own numerical scales.

```
Notebook Output — Cell 6
                Time            V1            V2            V3            V4  \
count  284807.000000  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05
mean    94813.859575  1.175161e-15  3.384974e-16 -1.379537e-15  2.094852e-15
std     47488.145955  1.958696e+00  1.651309e+00  1.516255e+00  1.415869e+00
min         0.000000 -5.640751e+01 -7.271573e+01 -4.832559e+01 -5.683171e+00
25%     54201.500000 -9.203734e-01 -5.985499e-01 -8.903648e-01 -8.486401e-01
50%     84692.000000  1.810880e-02  6.548556e-02  1.798463e-01 -1.984653e-02
75%    139320.500000  1.315642e+00  8.037239e-01  1.027196e+00  7.433413e-01
max    172792.000000  2.454930e+00  2.205773e+01  9.382558e+00  1.687534e+01

                 V5            V6            V7            V8            V9  \
count  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05
mean   1.021879e-15  1.494498e-15 -5.620335e-16  1.149614e-16 -2.414189e-15
std    1.380247e+00  1.332271e+00  1.237094e+00  1.194353e+00  1.098632e+00
min   -1.137433e+02 -2.616051e+01 -4.355724e+01 -7.321672e+01 -1.343407e+01
25%   -6.915971e-01 -7.682956e-01 -5.540759e-01 -2.086297e-01 -6.430976e-01
50%   -5.433583e-02 -2.741871e-01  4.010308e-02  2.235804e-02 -5.142873e-02
75%    6.119264e-01  3.985649e-01  5.704361e-01  3.273459e-01  5.971390e-01
max    3.480167e+01  7.330163e+01  1.205895e+02  2.000721e+01  1.559499e+01

       ...           V21           V22           V23           V24  \
count  ...  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05
mean   ...  1.628620e-16 -3.576577e-16  2.618565e-16  4.473914e-15
std    ...  7.345240e-01  7.257016e-01  6.244603e-01  6.056471e-01
min    ... -3.483038e+01 -1.093314e+01 -4.480774e+01 -2.836627e+00
25%    ... -2.283949e-01 -5.423504e-01 -1.618463e-01 -3.545861e-01
50%    ... -2.945017e-02  6.781943e-03 -1.119293e-02  4.097606e-02
75%    ...  1.863772e-01  5.285536e-01  1.476421e-01  4.395266e-01
max    ...  2.720284e+01  1.050309e+01  2.252841e+01  4.584549e+00

                V25           V26           V27           V28         Amount  \
count  2.848070e+05  2.848070e+05  2.848070e+05  2.84807
```

Figure 3.3: Descriptive Statistics of Transaction Features

Interpretation:

The statistics show that the variables have different scales and ranges. This supports the use of feature standardization before training the scale-sensitive Logistic Regression model.

3.4 Missing-Value Analysis

Missing-value analysis was performed using df.isnull().sum(). Every column reported a missing count of zero, resulting in a total of zero missing values across the dataset.

Figure 3.4: Missing-Value Analysis

Interpretation:

The dataset is complete with respect to missing observations, so no imputation or row deletion was required before modelling.

3.5 Class Distribution Analysis

The Class target was examined using value_counts(). There are 284,315 non-fraudulent transactions and 492 fraudulent transactions. Fraudulent transactions account for 0.1727% of the complete dataset, while non-fraudulent transactions account for 99.8273%.

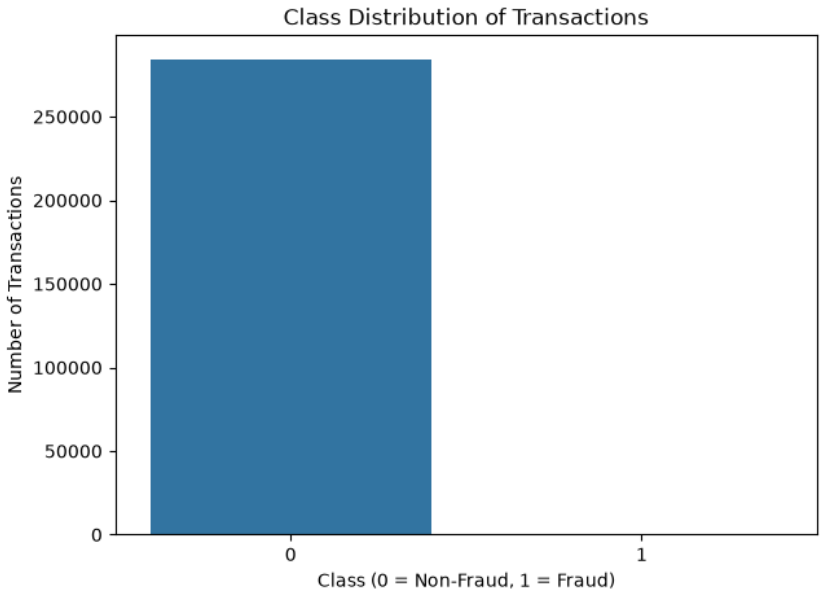

Figure 3.5: Class Distribution of Transactions

Interpretation:

The chart clearly demonstrates severe class imbalance. Because fraud is extremely rare, accuracy alone cannot represent the practical quality of the fraud detector.

3.6 Transaction Amount Distribution

The distribution of transaction Amount was examined separately for the two classes using density histograms. The visualization compares the amount distributions of fraudulent and non-fraudulent transactions.

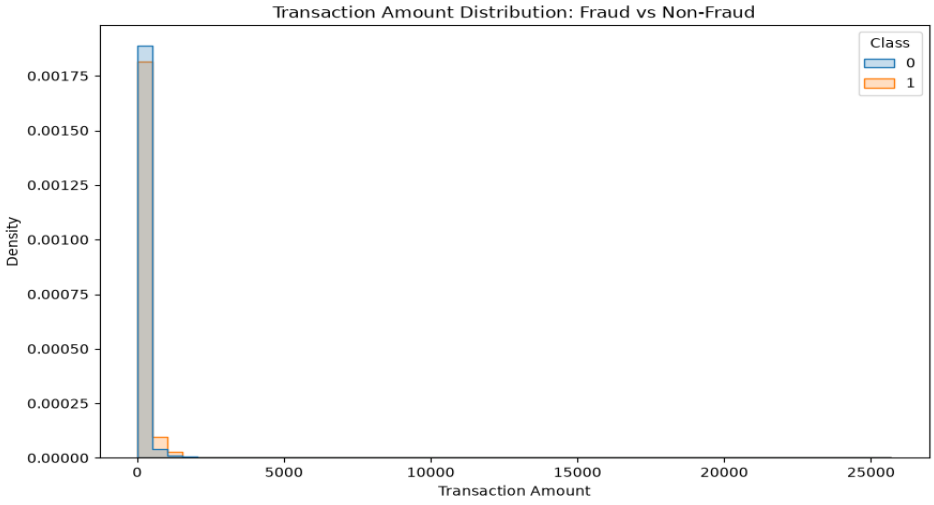

Figure 3.6: Transaction Amount Distribution – Fraud vs Non-Fraud

Interpretation:

The distributions are concentrated at lower transaction amounts, while the long right tail shows that a smaller number of transactions have substantially higher amounts. The two classes overlap, so Amount alone is not sufficient for reliable fraud detection.

3.7 Transaction Amount by Class

A boxplot was used to compare transaction amounts between non-fraudulent and fraudulent classes. The visualization highlights the central ranges and extreme observations in each class.

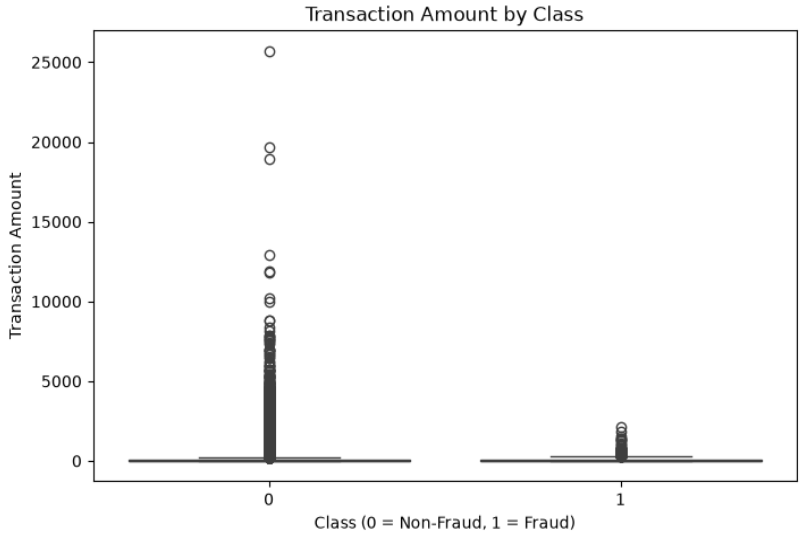

Figure 3.7: Transaction Amount by Class

Interpretation:

The plot shows many extreme amount values, particularly for the non-fraudulent class because it contains vastly more observations. Fraudulent transactions also contain higher-value observations, but the distributions overlap.

3.8 Fraud Percentage by Hour

The transaction Time value was converted into an Hour-of-Day feature using the number of elapsed seconds. The fraud percentage was then calculated for each hour by grouping transactions and taking the mean of the binary Class variable.

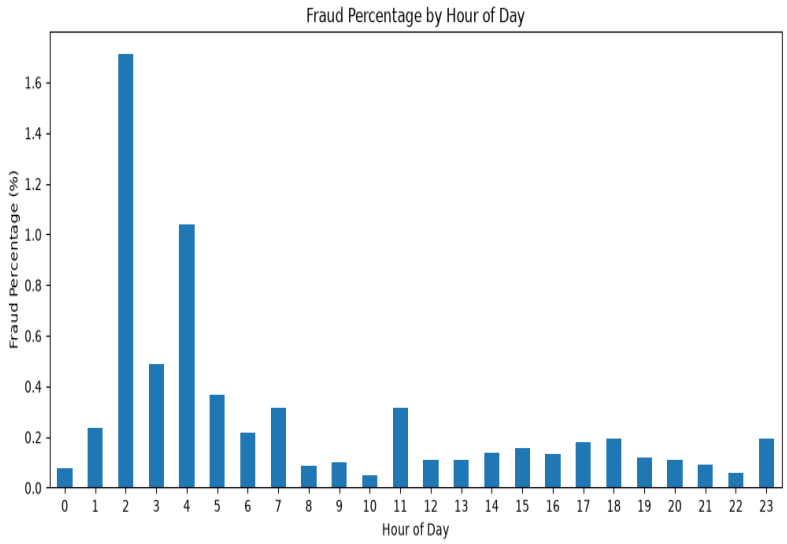

Figure 3.8: Fraud Percentage by Hour of Day

Interpretation:

The hourly fraud rate varies across the day. Some early-morning hours show higher percentages than several daytime hours, although the underlying transaction counts differ by hour. This makes the Hour feature useful for exploratory analysis.

3.9 Transaction Distribution by Hour

The number of transactions by hour was visualized with Class as the hue. This allows the overall transaction volume and the relative presence of fraudulent transactions to be observed across the 24-hour period.

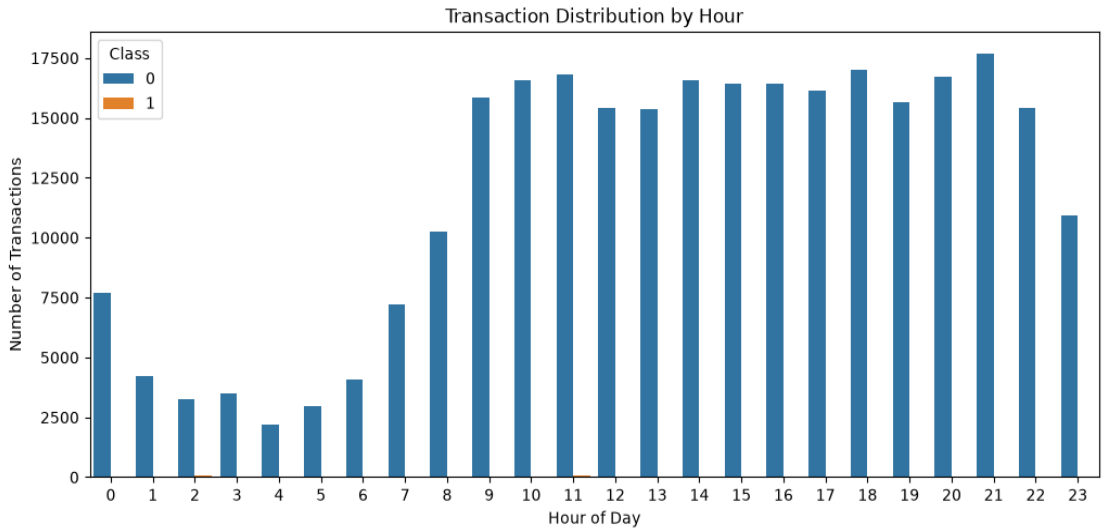

Figure 3.9: Transaction Distribution by Hour

Interpretation:

Transaction volume varies substantially across the hours. Non-fraudulent transactions dominate the count in every hour, while fraudulent transactions remain comparatively rare, consistent with the overall class imbalance.

3.10 Feature Preparation

The feature matrix X was created by removing Class and the derived Hour column. The resulting feature matrix contains 284,807 observations and 30 predictors, while y contains the 284,807 binary Class labels.

Figure 3.10: Feature and Target Shapes

Interpretation:

The output confirms that the modelling pipeline uses 30 input features and a separate binary target. Removing the target prevents data leakage during model training.

3.11 Stratified Train-Test Split

The dataset was divided into training and testing subsets using train_test_split() with test_size=0.20, random_state=42, and stratify=y. The split produced 227,845 training observations and 56,962 testing observations. The training set contains 227,451 non-fraudulent and 394 fraudulent transactions, while the test set contains 56,864 non-fraudulent and 98 fraudulent transactions.

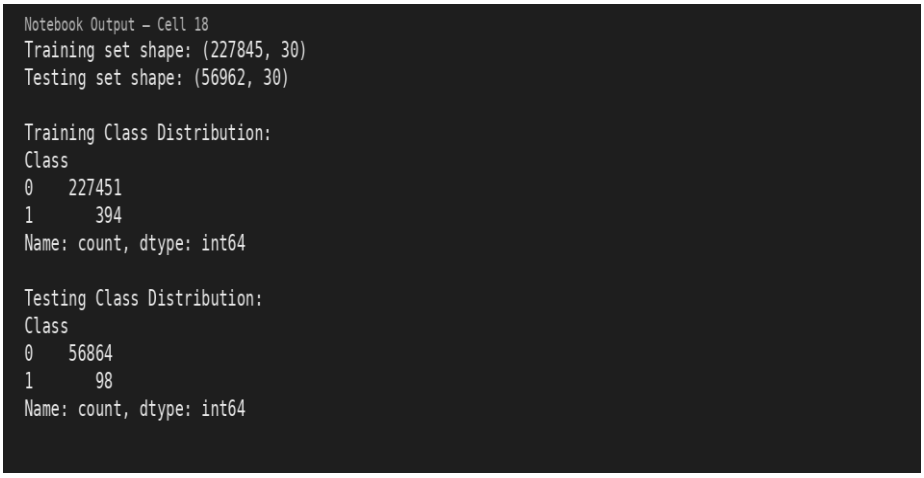

Figure 3.11: Stratified Training and Testing Split

Interpretation:

The 80:20 split provides an independent test set for generalization evaluation. Stratification preserves the rare fraud-class proportion in both subsets.

3.12 Feature Scaling

StandardScaler was fitted on the training features and then applied to both training and testing data. The scaled datasets retain 227,845 training rows and 56,962 testing rows, each with 30 features.

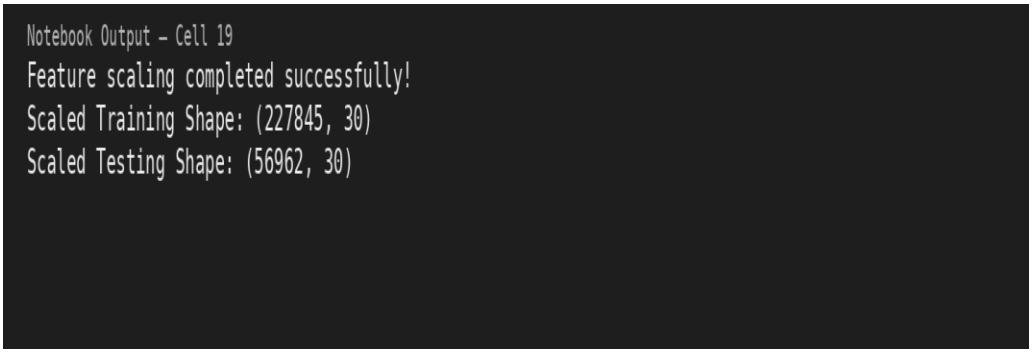

Figure 3.12: Feature Scaling Using StandardScaler

Interpretation:

The scaling step standardizes feature magnitudes and prevents variables with larger numerical ranges from dominating scale-sensitive classification algorithms.

3.13 SMOTE Class Balancing

SMOTE was applied only to the scaled training data. Before resampling, the training set contained 227,451 non-fraudulent samples and only 394 fraudulent samples. After SMOTE, both classes contained 227,451 samples.

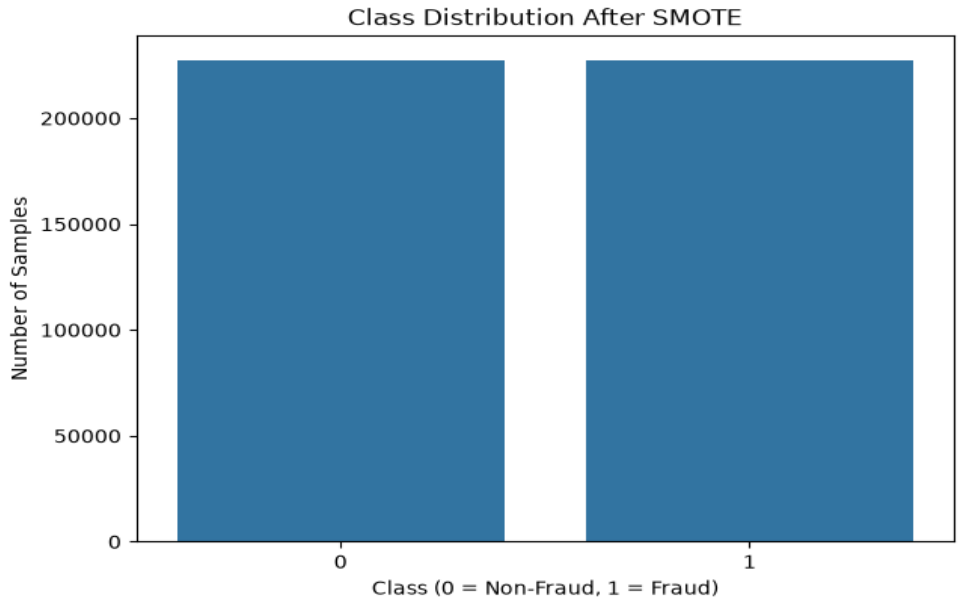

Figure 3.13: Class Distribution After SMOTE

Interpretation:

SMOTE creates a balanced training set by generating synthetic minority-class examples. Applying it only to the training data prevents synthetic observations from leaking into the test set.

3.14 Logistic Regression Model

Logistic Regression was trained on the SMOTE-balanced training data using max_iter=1000 and random_state=42. Predictions and fraud probabilities were then generated for the untouched scaled test set.

Figure 3.14: Logistic Regression Training

Interpretation:

The successful training output confirms that the Logistic Regression classifier completed the training stage on the balanced data.

3.15 Logistic Regression Predictions

After training, the Logistic Regression model generated class predictions and fraud probabilities for the test set. The predicted probabilities were retained for ROC-AUC and ROC-curve analysis.

Figure 3.15: Logistic Regression Predictions

Interpretation:

The prediction stage completed successfully and produced both binary predictions and probability estimates required for evaluation.

3.16 Random Forest Model

A RandomForestClassifier was trained with 50 trees, max_depth=15, random_state=42, and n_jobs=-1. The model was fitted on the SMOTE-balanced training data and then used to generate predictions and fraud probabilities for the test set.

Figure 3.16: Random Forest Training

Interpretation:

The Random Forest training completed successfully. Limiting the tree depth and using 50 estimators provided a practical model configuration for the large training set.

3.17 Random Forest Predictions and Evaluation

Random Forest predictions were generated successfully. On the test set, the model achieved 0.9988 accuracy, 0.6212 precision, 0.8367 recall, 0.7130 F1-score, and 0.9729 ROC-AUC. The corresponding classification report shows strong performance on both classes, with particularly meaningful performance on the rare fraud class.

```
Notebook Output — Cell 27
RANDOM FOREST
----------------------------
Accuracy  : 0.9988
Precision : 0.6212
Recall    : 0.8367
F1-Score  : 0.7130
ROC-AUC   : 0.9729

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56864
           1       0.62      0.84      0.71        98

    accuracy                           1.00     56962
   macro avg       0.81      0.92      0.86     56962
weighted avg       1.00      1.00      1.00     56962
```

Figure 3.17: Random Forest Evaluation Results

Interpretation:

The Random Forest model provides a substantially better balance between precision and recall than Logistic Regression. Its 62.12% precision means that a much larger share of fraud alerts are genuine compared with the Logistic Regression model.

3.18 Random Forest Confusion Matrix

The Random Forest confusion matrix contains 56,814 true negatives, 50 false positives, 16 false negatives, and 82 true positives on the 56,962 transaction test set.

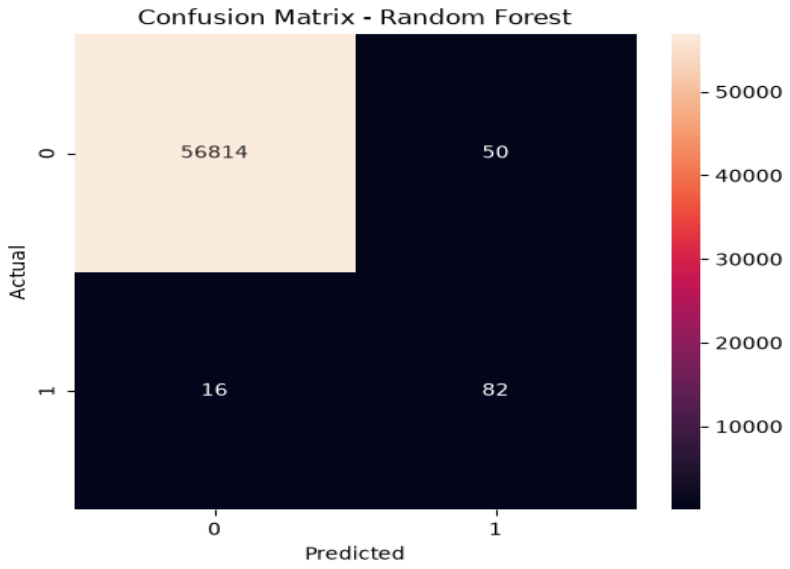

Figure 3.18: Confusion Matrix – Random Forest

Interpretation:

The matrix shows that 82 of the 98 actual fraudulent transactions were detected, while 16 fraudulent transactions were missed. Only 50 legitimate transactions were incorrectly flagged as fraud.

3.19 Logistic Regression Confusion Matrix

The Logistic Regression confusion matrix contains 55,397 true negatives, 1,467 false positives, 8 false negatives, and 90 true positives.

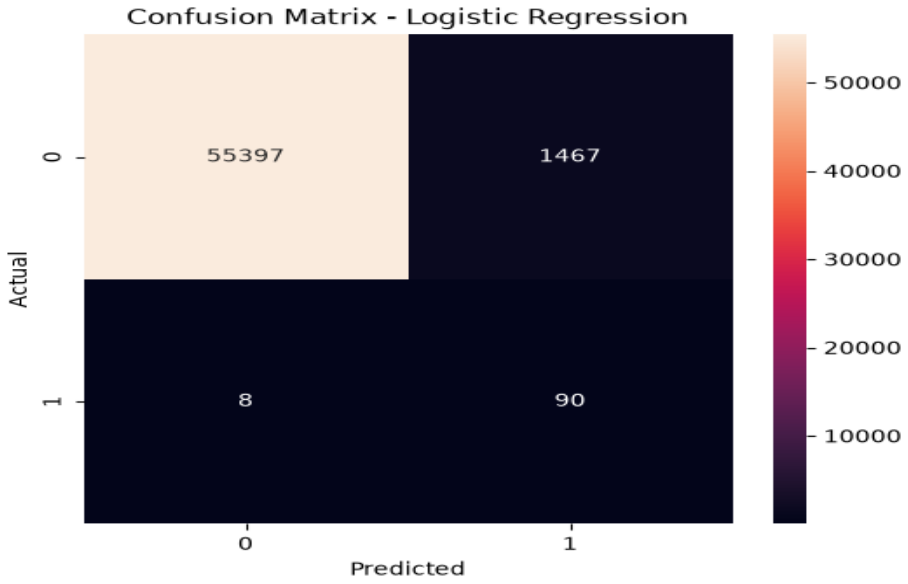

Figure 3.19: Confusion Matrix – Logistic Regression

Interpretation:

Logistic Regression detects more of the actual fraudulent transactions, but it generates many more false positives. This explains its high recall but very low precision.

3.20 Model Comparison

The two models were compared using accuracy, precision, recall, F1-score, and ROC-AUC. Logistic Regression achieved 0.974106 accuracy, 0.057803 precision, 0.918367 recall, 0.108761 F1-score, and 0.970843 ROC-AUC. Random Forest achieved 0.998841 accuracy, 0.621212 precision, 0.836735 recall, 0.713043 F1-score, and 0.972943 ROC-AUC.

Notebook Output – Cell 30

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression	0.974106	0.057803	0.918367	0.108761	0.970843
1	Random Forest	0.998841	0.621212	0.836735	0.713043	0.972943

Figure 3.20: Logistic Regression vs Random Forest – Evaluation Metrics

Interpretation:

Random Forest is the stronger overall model because it provides much higher precision and F1-score while retaining a high recall and slightly higher ROC-AUC. Logistic Regression has the higher recall, but its very low precision would produce a large number of false alarms.

3.21 ROC Curve Comparison

ROC curves were generated from the predicted fraud probabilities of both classifiers. The resulting AUC values were 0.971 for Logistic Regression and 0.973 for Random Forest.

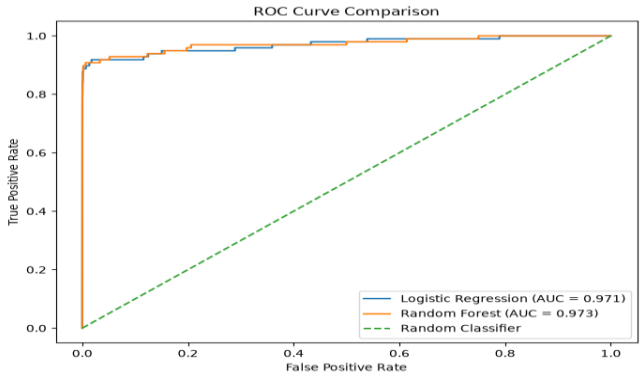

Figure 3.21: ROC Curve Comparison

Interpretation:

Both models separate fraudulent and non-fraudulent transactions effectively according to ROC-AUC. Random Forest has a small advantage, with an AUC of approximately 0.973.

3.22 Random Forest Feature Importance

Random Forest feature importance was calculated from the fitted model and the top 15 features were visualized. V14 had the highest importance at approximately 0.289, followed by V10, V4, V17, and V3.

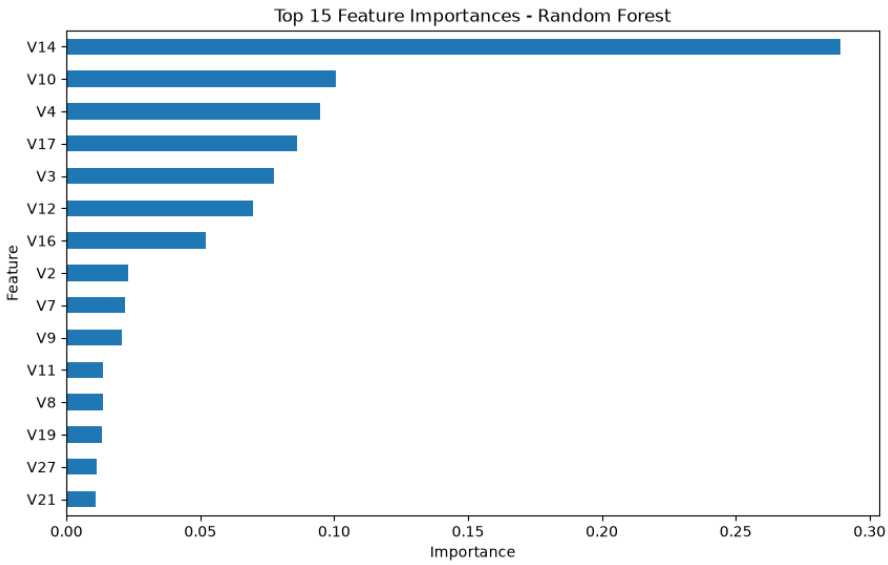

Figure 3.22: Top 15 Feature Importances – Random Forest

Interpretation:

V14 is the dominant feature in the fitted Random Forest, with an importance of approximately 0.289. V10, V4, V17, V3, and V12 also contribute substantially. These results help identify which anonymized transaction attributes the model relied on most heavily.

CHAPTER 4: RESULTS & DISCUSSION

4.1 Key Findings

1. The dataset contains 284,807 transactions, of which only 492 are fraudulent.
2. Fraud represents 0.1727% of all observations, creating a severe class-imbalance problem.
3. No missing values were found in the dataset.
4. Transaction amount distributions overlap between the two classes, so a single amount-based rule is insufficient.
5. Transaction volume changes by hour, and the hourly fraud percentage also varies.
6. An 80:20 stratified split preserved the rare fraud class in the test set.
7. SMOTE balanced the training data from 394 fraudulent observations to 227,451 synthetic/real minority-class training samples, matching the non-fraud class.
8. Logistic Regression achieved the highest recall at 91.84%, but its precision was only 5.78%.
9. Random Forest achieved 99.88% accuracy, 62.12% precision, 83.67% recall, 71.30% F1-score, and 0.9729 ROC-AUC.
10. Random Forest provided the best balance between detecting fraud and limiting false alerts.
11. V14 was the most important Random Forest feature, followed by V10, V4, V17, and V3.

4.2 Which Metric Matters Most for Fraud Detection?

Recall is particularly important in fraud detection because it measures the proportion of actual fraudulent transactions that are correctly identified. A false negative means that a fraudulent transaction is classified as legitimate, which can result in financial loss.

Precision is also important because a large number of false positives can cause legitimate transactions to be flagged as fraudulent, creating inconvenience for customers and increasing manual review effort.

In this project, Logistic Regression achieved a higher recall of 91.84%, while Random Forest achieved a recall of 83.67%. However, Logistic Regression had very low precision of 5.78%, whereas Random Forest achieved 62.12% precision. Random Forest also achieved a much higher F1-score of 71.30% compared with 10.88% for Logistic Regression.

Therefore, recall is highly important when the main objective is to minimize missed fraud cases, but precision must also be considered to control false alarms. For this project, Random Forest provides a better balance between precision and recall.

4.3 Scalability: Handling 1 Million Transactions per Hour

A system processing 1 million transactions per hour would need to handle approximately 278 transactions per second. To support this workload, the fraud-detection model could be deployed as a real-time prediction service with batch or streaming processing.

Transaction data could be received through a distributed streaming system, while the trained model could process transactions in parallel. Random Forest prediction can be parallelized, and the model-serving infrastructure can be scaled horizontally as transaction volume increases.

The production system should monitor prediction latency, throughput, false positives, and false negatives. Suspicious transactions can be sent for further investigation, while prediction results can be stored for auditing and analysis. Periodic retraining would be useful because fraud patterns can change over time.

4.4 Conclusion

The Fraud Detection project successfully implemented a complete machine-learning workflow for a severely imbalanced financial transaction dataset. The analysis established the importance of class-aware evaluation and showed why accuracy alone is misleading in fraud detection.

The Random Forest model was the strongest overall performer in the implemented comparison. It achieved 99.88% accuracy, 62.12% precision, 83.67% recall, 71.30% F1-score, and 0.9729 ROC-AUC. Although Logistic Regression obtained a higher recall of 91.84%, its precision of 5.78% resulted in a much larger number of false fraud alerts.

The final workflow demonstrates how preprocessing, stratified splitting, feature scaling, SMOTE, classification modelling, confusion-matrix analysis, ROC-AUC evaluation, and feature-importance analysis can be combined to build a practical fraud-detection pipeline.

4.5 Future Enhancements

1. Evaluate additional algorithms such as Gradient Boosting, XGBoost, LightGBM, or calibrated ensemble methods.
2. Use cross-validation and hyperparameter tuning to optimize precision-recall trade-offs.
3. Experiment with threshold tuning because the default classification threshold may not be optimal for fraud operations.
4. Monitor model drift and retrain periodically as fraud patterns evolve.
5. Deploy the model as a scalable real-time service for high-volume transaction processing.

6. Add operational monitoring for latency, false-positive rate, false-negative rate, and investigation outcomes.

4.6 References

1. OASIS INFOBYTE – Data Analytics Internship, Level 2 Task 3: Fraud Detection task requirements and implementation workflow.
2. Credit Card Fraud Detection Dataset used in the submitted Jupyter Notebook.
3. Scikit-learn documentation and APIs used for train-test splitting, feature scaling, Logistic Regression, Random Forest, evaluation metrics, and ROC analysis.
4. Imbalanced-learn documentation for SMOTE-based minority-class oversampling.
5. Pandas, NumPy, Matplotlib, and Seaborn libraries used for data handling and visualization.

4.7 Final Findings and Conclusion

The implemented pipeline successfully transformed the raw credit-card transaction data into a balanced training workflow and produced reliable test-set evaluation results. The final comparison supports Random Forest as the preferred model for this implementation because it combines high recall with substantially better precision and F1-score than Logistic Regression. The project also demonstrates that fraud detection should be treated as a class-imbalance problem where precision, recall, F1-score, ROC-AUC, and confusion-matrix outcomes are central to model selection.